Informe — TP N.º 8

Food Store en la feria: la base de la tablet y el cierre de la capa de datos

Alumno: Saleme Ismael · **Asignatura:** Base de Datos II (UTN — TUP a Distancia) **Motores:** PostgreSQL 17.10 (`Food_Store_Copia`) · SQLite 3.50.4 · MongoDB (Unidad 7) **Fecha:** 2026-11-01

1.a — Qué datos viajan a la tablet

La regla que decide qué viaja es una sola: **viaja lo que el vendedor necesita para armar un pedido, y nada más**. Todo lo demás se queda en el servidor. La tablet guarda una copia fechada y la trata como una foto que puede estar vieja.

Columna del servidor	¿Viaja?	Por qué
`producto.id`	Sí	Es la identidad que comparten los dos motores. Sin ella, el borrador no se puede cruzar contra el servidor.
`producto.nombre`	Sí	El vendedor tiene que poder nombrar el producto en pantalla.
`producto.precio` -> `precio_centavos`	Sí	Es el precio que el cliente ve. Se convierte a entero para no arrastrar punto flotante al dinero.
`producto.stock` -> `stock_visto`	Sí	El vendedor necesita saber cuánto hay para no ofrecer de más. Es un valor **observado**, no reservado.
`activo AND stock > 0` -> `disponible`	Sí	Traducido: el vendedor no tiene que razonar sobre dos columnas para saber si puede vender.
`NOT activo` -> `eliminado`	Sí	Traducido, y la fila **no se borra**: si se borrara, los borradores históricos perderían su clave foránea.
`now()` -> `fecha_copia`	Sí	La sella el servidor. Si la pusiera la tablet, el dato que sirve para juzgar si la copia envejeció sería el menos confiable.
`cliente` (nombre, email, teléfono)	**No**	El vendedor no los necesita para cobrar, y son datos personales que no tienen por qué estar en un dispositivo que se pierde en una feria.
`usuario` (password_hash, rol)	**No**	La tablet no se autentica contra PostgreSQL: no hay credenciales en el dispositivo.
`pedido` / `detalle_pedido`	**No**	Es el historial de ventas. La tablet sólo manda propuestas; no tiene tabla de ventas a propósito.
`categoria_id`	**No**	La tablet no navega por categorías. Si se quisiera un filtro por rubro, se agregaría.
`lote`, `deposito`, `control_lote`	**No**	Es logística interna, no venta.

Quién es la fuente de verdad: PostgreSQL, siempre. La tablet no puede confirmar precios ni descontar stock; su mejor resultado posible se llama `SIMULADO_APROBADO`, y no es una venta. El único canal entre los dos motores es un CSV de una sola dirección: la tablet no tiene conexión, ni usuario, ni driver.

2.c — Los casos: preparación, resultado y explicación

Cada caso se prepara **cambiando datos en la copia del servidor**, exportando una instantánea nueva y corriendo el semáforo. La secuencia completa está automatizada en `movil/simular_casos.ps1`; la evidencia cruda, en `movil/evidencias_casos.txt`.

Caso	Qué preparé	Resultado real
C5	Borrador con producto 2 (×2) y producto 4 (×1), al precio vigente.	`SIMULADO_APROBADO: todo coincide (simulación, no es una venta)`
C3	Borrador del producto 95 con `stock_visto = 3`; se piden **10** unidades.	`RECHAZADO: producto 95: no alcanza el stock (pide 10, hay 3)`
C2a	Borrador del producto 5 a 111923 centavos. Después, `psql -f db/tp8_caso_c2_sube_precio.sql` (1119.23 -> 1269.23), nueva instantánea y semáforo.	`CONFLICTO: producto 5: precio cambió de 111923 a 126923 centavos`
C2b	El cliente acepta: se actualiza el precio visto a 126923 y se encola una solicitud **nueva** (UUID 2b…) sobre el **mismo** borrador.	`SIMULADO_APROBADO`. La solicitud vieja queda en `CONFLICTO` como registro histórico.
C4	Borrador del producto 6. Después, `psql -f db/tp8_caso_c4_baja_logica.sql` (`activo = false`), nueva instantánea y semáforo.	`RECHAZADO: producto 6: dado de baja`. En `producto_cache` la fila queda con `disponible = 0`, `eliminado = 1` y sale del catálogo.
C6	Reenvío de la solicitud `50000000-…-000000000005`, que ya existe.	`UNIQUE constraint failed: cola_sync.solicitud_uuid`. Repetir el semáforo no reevalúa nada: los cinco estados quedan idénticos.

Por qué cada uno da lo que da. En **C5** no hay nada que resolver: verde no significa "vendido", significa "si esto se enviara ahora, el servidor lo habría aceptado". En **C3** el producto existe y está disponible, así que el rojo no es de catálogo sino de cantidad: la tablet no puede saber si esas 3 unidades siguen ahí. En **C2** no hay rojo —el producto existe, hay stock— pero el cliente vio un precio que ya no es el del servidor: eso es amarillo, y lo decide una persona. En **C4** la baja es lógica; si se

borrara la fila, el borrador no podría distinguir "dado de baja" de "nunca existió", que son dos incidentes distintos. En **C6** la clave primaria impide duplicar la solicitud, pero esa protección es del lado del cliente: si la tablet se reinicia y genera otro UUID, no la detecta.

Prueba de que PostgreSQL no cambió

Al final se corre `psql -f db/tp8_verificacion_final.sql`, que compara contra la línea de base tomada **antes** de empezar (`db/tp8_linea_base.sql`):

Invariante	Antes	Después	Veredicto
`pedido` (filas)	200.000	200.000	SIN CAMBIOS
`detalle_pedido` (filas)	200.000	200.000	SIN CAMBIOS
`stock_total`	4.999.032	4.999.032	SIN CAMBIOS
`stock_huella` (md5 del vector `id:stock`)	`95b86fa0…`	`95b86fa0…`	SIN CAMBIOS
`precio_huella` (md5 del vector `id:precio`)	`1816a7c5…`	`c47b59be…`	CAMBIÓ (esperado, C2)

La última fila es la que vale: **el precio cambió y el stock no**. Se usa la huella md5 y no sólo la suma porque dos cambios opuestos —uno sube 3, otro baja 3— dejan la suma igual; el md5 del vector completo detecta cualquier cambio en cualquier fila. Que el precio se haya movido y el stock no es la demostración de que el semáforo **leyó y no escribió**.

3.a — Mapa de la capa de datos

Motor	Qué guarda	Quién manda	Qué no puede hacer
PostgreSQL	El catálogo completo (50.000 productos, 49.744 vendibles), 20.000 clientes, 200.000 pedidos y sus detalles, stock, precios, categorías, lotes y depósitos.	**Es la fuente de verdad.** Manda sobre precio, stock, existencia y toda venta.	—
MongoDB (U7)	Pedidos desnormalizados: `_id` = `pedido.id`, cliente embebido, líneas embebidas, `total` congelado al momento de la venta.	Manda sobre la **forma del documento**, no sobre el dato. Si discrepa con PostgreSQL, gana PostgreSQL.	No origina ni corrige precios, stock o ventas.
SQLite (tablet)	Copia fechada del catálogo + borradores + cola de solicitudes.	Manda sobre los **borradores sin confirmar**: son suyos y nadie más los ve.	No manda sobre precio, stock ni ventas. No tiene conexión a PostgreSQL.

Quién manda sobre cada dato

Dato	PostgreSQL	MongoDB	SQLite
Precio	Manda.	Lo **congela** al momento de la venta; no se actualiza.	Lo **copia** como `precio_centavos_visto`: es lo que el cliente vio.
Stock	Manda. Es el único que descuenta.	No lo guarda: no le corresponde.	Lo copia como `stock_visto`: una observación, no una reserva.
Identidad del pedido	Asigna el `id` definitivo.	Conserva ese `id` en `_id`.	Crea el `id_local` (UUID) mientras el pedido no existe en el servidor.
Venta confirmada	La crea, en una transacción.	La refleja como documento de lectura.	**No la tiene.** No hay tabla de ventas.
Baja de un producto	Manda: `activo = false`.	La refleja si el pedido se proyecta después.	La copia como `eliminado = 1` y **conserva la fila**.

En una frase: **PostgreSQL decide, MongoDB recuerda cómo se veía, SQLite propone.**

3.b — Plan de mantenimiento

Qué se respalda y cómo se prueba la restauración

- **PostgreSQL:** `pg_dump` diario (formato custom) + retención de 30 días. La prueba no es que el archivo exista: es **restaurarlo en una base distinta** (`Food_Store_Restore`) y comparar las huellas (`count(*)` de pedidos, md5 del vector de stock) contra la base viva. Si coinciden, el respaldo sirve. Un respaldo que nunca se restauró no es un respaldo: es una carpeta.
- **SQLite (tablet):** no se respalda el `.db` como fuente de verdad —no lo es—. Se respalda **la cola**: los borradores no sincronizados son lo único que no se puede reconstruir desde el servidor. Al cerrar la feria se exportan `pedido_borrador`, `item_borrador` y `cola_sync`.
- **MongoDB:** `mongodump` diario, restauración en una instancia aparte y comparación de `count` por colección. Si PostgreSQL se restaura, la proyección de Mongo queda desfasada y se regenera: es derivada, no fuente.
- **Registro:** cada prueba de restauración se anota con fecha, base de destino y resultado de las huellas. Sin registro, no se sabe si el plan funciona.

Si se restaura un respaldo viejo de PostgreSQL, ¿qué hacen las tablets?

Es la situación más peligrosa del sistema, porque **invierte quién sabe más**: la tablet tiene una instantánea con `fecha_copia` posterior al punto de restauración, así que cree estar más actualizada que el servidor.

- Un borrador armado después del respaldo compara contra precios que el servidor ya no tiene: aparece un `CONFLICTO` masivo.
- Un producto dado de baja después del respaldo **reaparece como activo**. Si el semáforo se corre contra esa instantánea vieja, puede dar verde a algo que ya no se vende.
- Si se confirmara una venta, se descontaría stock de un estado que ya se perdió: se vendería dos veces el mismo producto.

Qué deben hacer: ninguna tablet envía hasta que el servidor publique una instantánea nueva con `fecha_copia` posterior al restore. El servidor marca una época nueva y rechaza solicitudes cuya instantánea sea anterior. La tablet, al ver una `fecha_copia` más nueva, recarga la cache **completa** (no incremental), conserva los borradores y los **revalida**: lo que estaba en `SIMULADO_APROBADO` vuelve a `PENDIENTE`. Por eso `fecha_copia` la sella el servidor: es el dato que permite detectar esta situación.

¿Cada cuánto y quién revisa las solicitudes en CONFLICTO o RECHAZADO?

- **CONFLICTO:** lo resuelve el **responsable de caja en el momento**, porque el cliente está enfrente y hay que decidir si acepta el precio nuevo. Si no se resuelve en el día, pasa al responsable del food truck al cierre.
- **RECHAZADO:** no requiere decisión de precio; se revisa **al cierre de cada jornada** para detectar patrones. Un producto que se rechaza siempre es una señal: stock mal cargado, o un producto discontinuado que sigue en la cache.
- **Frecuencia:** cierre diario de la feria + revisión semanal de patrones.
- **Quién más:** nadie. La cola es de la tablet; el servidor no la ve.

3.c — Code review con IA

La revisión se hizo con OpenCode en modo **Plan** sobre los scripts de `db/`. De los hallazgos encontrados se desarrollan cuatro, de tres unidades distintas. Cada uno se comprobó **ejecutando** (`db/tp8_code_review_verificaciones.sql`, salida en `docs/evidencias_code_review.txt`), no leyendo el script.

#	Unidad	Hallazgo	Decisión	Motivo	Cómo lo comprobé
H1	U3	El índice `pedido(forma_pago)` se descartó diciendo que "el optimizador hará Seq Scan igualmente".	**Descartado** (con fundamento corregido)	El planner **sí** usa el índice: `Bitmap Index Scan`. Pero lee **las 1471 páginas igual**, porque el 33% de las filas está en casi todas las páginas. No ahorra I/O; sólo agrega costo de escritura.	Cardinalidad real: 33,53 / 33,29 / 33,18 %. Sin índice: Seq Scan, 9,748 ms. Con índice: Bitmap Heap Scan, `Heap Blocks: exact=1471`. Se creó dentro de una transacción y se revirtió: 0 índices quedaron.
H2	U5	`REVOKE SELECT (precio)` no recorta un `GRANT SELECT` de tabla.	**Corregido**	Un `REVOKE` por columna no puede quitar lo que un `GRANT` de tabla otorgó. La forma correcta es revocar el privilegio de tabla y volver a otorgar **por columnas**.	`has_column_privilege(...)` siguió en `true` después del `REVOKE` por columna. Con `REVOKE ALL` + `GRANT SELECT (id, nombre, stock, activo, categoria_id)` dio `false`. `roles.sql` ya usa `GRANT` por columnas.
H3	U5	Un `GRANT` dentro de un bloque `DO` se ejecuta con los privilegios del invocador.	**Corregido**	Un `DO` no cambia de usuario: si lo corre un superusuario, el `GRANT` se aplica **de verdad**. Una prueba de seguridad que usa `DO` para "intentar" un `GRANT` no prueba nada.	Se crearon dos roles; el `DO` otorgó el rol; `pg_has_role(...)` dio `true`. Se revocó y se eliminaron los roles: 0 roles quedaron.
H4	U1–U3	No existe ninguna regla de stock suficiente en la base.	**Aceptado** (deuda documentada)	Se pueden vender 999 unidades de un producto con stock 3: no hay `CHECK`, ni trigger, ni procedimiento que lo impida. El semáforo de la tablet lo detecta, pero **la fuente de verdad no**.	`INSERT` de 999 unidades en una transacción: entró sin error. `ROLLBACK`; el stock quedó intacto.

Por qué H4 queda como deuda y no como corrección aplicada. Un trigger que compare `cantidad` contra `stock` sin bloquear la fila es una condición de carrera en espera: dos cajas pueden leer el mismo stock 1 y venderlo las dos. La corrección correcta necesita `SELECT … FOR UPDATE` dentro de la transacción de venta, y además hay que decidir si el `INSERT` descuenta stock o no. Eso cambia el contrato del servidor y merece su propia revisión; aplicarlo de apuro en este TP sería peor que documentarlo.

Cierre de la revisión: nada quedó modificado — 0 índices `tp8`, 0 roles `tp8`, 200.000 pedidos. Los cuatro hallazgos son verificables con un comando cada uno, y ese es el punto: **lo que no se ejecutó no se sabe.**

Cierre

1. **La tablet no puede confirmar nada.** No tiene conexión, credenciales ni driver. El único canal es un CSV de una sola dirección.
2. **El semáforo distingue cuatro rojos y un amarillo**, en orden de precedencia, y el amarillo sólo aparece si no hay ningún rojo.
3. **PostgreSQL no se movió**: pedidos y stock idénticos a la línea de base, con huella md5. El precio sí cambió, y eso es lo que prueba que se leyó sin escribir.
4. **La idempotencia de la tablet no es la protección real.** La clave primaria de `cola_sync` protege la tabla local; la protección de negocio vive en el servidor, dentro de la transacción que crea el pedido.
5. **La fuente de verdad no impide vender sin stock.** El semáforo tapa el síntoma en la tablet; el problema sigue en PostgreSQL (H4).

Evidencias: dónde está cada cosa

La cátedra descuenta "entregar capturas sin scripts". Por eso cada afirmación de este informe tiene un archivo y un comando que la reproduce.

Qué se afirma	Archivo	Cómo se reproduce
La tablet crea su base desde cero, con FKs activas y 4 tablas `STRICT`.	`movil/schema_sqlite.sql`	`sqlite3 foodstore_movil.db ".read movil/schema_sqlite.sql"`
Las restricciones y las transacciones funcionan (COMMIT, ROLLBACK, C1, C7).	`movil/pruebas_sqlite.sql`	`sqlite3 pruebas_aparte.db ".read movil/pruebas_sqlite.sql"` (base aparte: borra las tablas)
La instantánea se exporta desde PostgreSQL y se carga en la tablet.	`db/tp8_v_snapshot_movil.sql`, `movil/schema_snapshot.sql`, `movil/cargar_snapshot.sql`	pasos 3, 6 y 7 del `README.md`
Los casos C2 a C6 dan el resultado esperado.	`movil/casos/`, `movil/simular_casos.ps1`	`.\movil\simular_casos.ps1 -> movil/evidencias_casos.txt`
PostgreSQL no cambió: pedidos y stock idénticos.	`db/tp8_linea_base.sql`, `db/tp8_verificacion_final.sql`	`psql -f db/tp8_verificacion_final.sql`
Los cuatro hallazgos del review, comprobados en la base.	`db/tp8_code_review_verificaciones.sql`	`psql -f … -> docs/evidencias_code_review.txt`
La spec de Kiro: requisitos, diseño y tareas.	`docs/kiro/requirements.md`, `design.md`, `tasks.md`	lectura directa
El orden completo de creación desde cero.	`README.md`	sección "Orden de creación desde cero"